# Credential cache

A credential cache (or "ccache") holds Kerberos credentials while they remain valid and, generally, while the user's session lasts, so that authenticating to a service multiple times (e.g., connecting to a web or mail server more than once) doesn't require contacting the KDC every time.

A credential cache usually contains one initial ticket which is obtained using a password or another form of identity verification. If this ticket is a ticket-granting ticket, it can be used to obtain additional credentials without the password. Because the credential cache does not store the password, less long-term damage can be done to the user's account if the machine is compromised.

A credentials cache stores a default client principal name, set when the cache is created. This is the name shown at the top of the klist *-A* output.

Each normal cache entry includes a service principal name, a client principal name (which, in some ccache types, need not be the same as the default), lifetime information, and flags, along with the credential itself. There are also other entries, indicated by special names, that store additional information.

## ccache types

The credential cache interface, like the keytab and replay cache interfaces, uses *TYPE:value* strings to indicate the type of credential cache and any associated cache naming data to use.

There are several kinds of credentials cache supported in the MIT Kerberos library. Not all are supported on every platform. In most cases, it should be correct to use the default type built into the library.

1. **API** is only implemented on Windows. It communicates

On this page

Credential cache
ccache types
Collections of caches
Tool alterations to use cache collection
Default ccache name

Table of contents

For users
For administrators
For application developers
For plugin module developers
Building Kerberos V5
Kerberos V5 concepts
Credential cache
keytab
replay cache

with a server process that holds the credentials in memory for the user, rather than writing them to disk.

2. **DIR** points to the storage location of the collection of the credential caches in *FILE:* format. It is most useful when dealing with multiple Kerberos realms and KDCs. For release 1.10 the directory must already exist. In post-1.10 releases the requirement is for parent directory to exist and the current process must have permissions to create the directory if it does not exist. See Collections of caches for details. New in release 1.10. The following residual forms are supported:

   - DIR:dirname
   - DIR::dirpath/filename - a single cache within the directory

   Switching to a ccache of the latter type causes it to become the primary for the directory.

3. **FILE** caches are the simplest and most portable. A simple flat file format is used to store one credential after another. This is the default ccache type if no type is specified in a ccache name.

4. **KCM** caches work by contacting a daemon process called `kcm` to perform cache operations. If the cache name is just `KCM:`, the default cache as determined by the KCM daemon will be used. Newly created caches must generally be named `KCM:uid:name`, where *uid* is the effective user ID of the running process.

   KCM client support is new in release 1.13. A KCM daemon has not yet been implemented in MIT krb5, but the client will interoperate with the KCM daemon implemented by Heimdal. macOS 10.7 and higher provides a KCM daemon as part of the operating system, and the **KCM** cache type is used as the default cache on that platform in a default build.

5. **KEYRING** is Linux-specific, and uses the kernel keyring support to store credential data in unswappable kernel memory where only the current user should be able to access it. The following residual forms are supported:

   - KEYRING:name

stash file
Supported date and time formats
Protocols and file formats
MIT Kerberos features
How to build this documentation from the source
Contributing to the MIT Kerberos Documentation
Resources

Full Table of Contents

Search

Go

- KEYRING:process:name - process keyring
- KEYRING:thread:name - thread keyring

Starting with release 1.12 the *KEYRING* type supports collections. The following new residual forms were added:

- KEYRING:session:name - session keyring
- KEYRING:user:name - user keyring
- KEYRING:persistent:uidnumber - persistent per-UID collection. Unlike the user keyring, this collection survives after the user logs out, until the cache credentials expire. This type of ccache requires support from the kernel; otherwise, it will fall back to the user keyring.

See Collections of caches for details.

6. **MEMORY** caches are for storage of credentials that don't need to be made available outside of the current process. For example, a memory ccache is used by kadmin to store the administrative ticket used to contact the admin server. Memory ccaches are faster than file ccaches and are automatically destroyed when the process exits.
7. **MSLSA** is a Windows-specific cache type that accesses the Windows credential store.

## Collections of caches

Some credential cache types can support collections of multiple caches. One of the caches in the collection is designated as the *primary* and will be used when the collection is resolved as a cache. When a collection-enabled cache type is the default cache for a process, applications can search the specified collection for a specific client principal, and GSSAPI applications will automatically select between the caches in the collection based on criteria such as the target service realm.

Credential cache collections are new in release 1.10, with support from the **DIR** and **API** ccache types. Starting in release 1.12, collections are also supported by the **KEYRING**

ccache type. Collections are supported by the **KCM** ccache type in release 1.13.

### Tool alterations to use cache collection

- kdestroy *-A* will destroy all caches in the collection.
- If the default cache type supports switching, kinit *princname* will search the collection for a matching cache and store credentials there, or will store credentials in a new unique cache of the default type if no existing cache for the principal exists. Either way, kinit will switch to the selected cache.
- klist *-l* will list the caches in the collection.
- klist *-A* will show the content of all caches in the collection.
- kswitch *-p princname* will search the collection for a matching cache and switch to it.
- kswitch *-c cachename* will switch to a specified cache.

## Default ccache name

The default credential cache name is determined by the following, in descending order of priority:

1. The **KRB5CCNAME** environment variable. For example, `KRB5CCNAME=DIR:/mydir/`.
2. The **default_ccache_name** profile variable in [libdefaults].
3. The hardcoded default, DEFCCNAME.


© Copyright 1985-2025, MIT.